



PostgreSQL Database Fundamentals

Eng. Fatma Elraey

Day 1 — From Database Concepts to Your First PostgreSQL Queries

Introduction to databases • PostgreSQL and pgAdmin • Creating databases and tables • DDL
and DML • Basic SQL queries • Filtering data

Section 1: Introduction and Database Fundamentals

Day 1 Learning Objectives

By the end of Day 1, participants should be able to:

- Define data, databases, and database management systems.
- Identify tables, rows, columns, cells, and primary keys.
- Explain common relationships between tables.
- Differentiate PostgreSQL, pgAdmin, and psql.
- Install and connect to a PostgreSQL server.
- Create and delete databases.
- Create and inspect tables.
- Insert, retrieve, update, and delete rows.
- Filter data using SQL operators.
- Use PostgreSQL through pgAdmin and SQL Shell.

What Is Data?

Data represents raw facts or values that can be stored and processed.

Examples

- A customer's name
- A telephone number
- A product price
- An employee's salary
- A student's date of birth
- The status of an order

Data becomes useful when it is:

- Organized
- Stored correctly
- Easy to retrieve
- Protected from unauthorized access
- Available when applications need it

✎ *note: a single value such as “Ahmed” is data. When we store Ahmed's name, telephone number, department, and salary together, they become a meaningful employee record.*

What Is a Database?

A database is an organized collection of related data that can be stored, searched, updated, and managed.

Common uses of databases

- Websites
- Mobile applications
- Banking systems
- Hospital systems
- School-management systems
- E-commerce platforms
- Human-resources systems
- Inventory systems

Example

An online store may use a database to store:

- Customers
- Products
- Orders
- Payments
- Deliveries

! *Important clarification: not every small program requires a database, but most applications that store dynamic, persistent, or shared information need one.*

Why Do We Use Databases?

Databases help applications:

- Store information permanently.
- Retrieve information quickly.
- Update existing information.
- Delete information that is no longer required.

- Reduce unnecessary duplication.
- Apply validation rules.
- Protect sensitive information.
- Support multiple users.
- Maintain relationships between different types of data.
- Recover information after certain failures.

 *Example: without a database, an online store may lose its product and order information whenever the application is restarted.*

Relational Databases

A relational database organizes data into tables. Each table represents an entity or subject.

Table	Information Stored
customers	Customer information
products	Product information
orders	Customer orders
employees	Employee information
departments	Department information

Tables can be connected through relationships.

Anatomy of a Database Table

Example: employees table

employee_id	first_name	last_name	phone
1	Fatma	Elraey	01020304050
2	Ahmed	Hassan	01130405060
3	Sara	Ali	01220304050

Table components

- Table name: employees
- Field or column: first_name

- Field name: the name written at the top of a column
- Record or row: all information about one employee
- Data item or cell: one value at the intersection of a row and column
- Key field: a column used to identify a particular row

Visual direction — highlight:

- One complete horizontal row as a record
- One complete vertical column as a field
- One individual value as a cell
- The employee_id column as the key field

Columns, Rows, and Cells

Columns define the structure

When creating a table, we define:

- Column name
- Data type
- Whether a value is required
- Default value
- Validation rules
- Key constraints

Rows contain the actual data

Every time we insert new information, PostgreSQL adds a new row.

Cells contain individual values

Each cell contains one value belonging to one row and one column. In the row:

● ● ● SQL				
1		Fatma		Elraey 01020304050

- 1 is the employee ID.
- Fatma is the first name.
- Elraey is the last name.
- 01020304050 is the telephone number.

Data Types

Each table column must have a data type. The data type determines the kind of value that can be stored.

Data Type	Example Use
smallint	Small whole numbers
integer	IDs, ages, quantities
bigint	Very large whole numbers
numeric(10,2)	Salaries and prices
varchar(100)	Names and limited text
text	Variable-length text
boolean	True or false values
date	Date only
time	Time only
timestamp	Date and time
bytea	Binary data

 PostgreSQL provides many native data types, including numeric, character, binary, Boolean, date, time, UUID, JSON, array, and network types.

Choosing the Correct Data Type

Use numeric types for values used in calculations

- Age
- Salary
- Product quantity
- Product price

Use text types for identifiers that are not calculated

- Telephone numbers
- National IDs
- Postal codes

- Employee codes

Why should a telephone number be text?

A telephone number may contain a leading zero, a country code such as +20, spaces, hyphens, or parentheses.

Correct example

```
SQL
phone varchar(20)
```

Incorrect example

```
SQL
phone integer
```

Storing 01020304050 as an integer removes its leading zero.

What Is a Primary Key?

A primary key is a column, or combination of columns, that uniquely identifies every row in a table.

Primary-key rules

A primary key must:

- Be unique.
- Not contain NULL.
- Identify exactly one row.
- Remain stable whenever possible.

Example

employee_id	first_name
1	Fatma
2	Ahmed
3	Sara

employee_id is the primary key because every employee has a different ID.

Why Use an ID Column?

Names are not always unique. Two employees may have the same name:

employee_id	first_name	last_name
15	Ahmed	Hassan
29	Ahmed	Hassan

The ID allows us to identify the correct employee.

SQL

```
DELETE FROM employees
WHERE employee_id = 29;
```

This is safer than deleting by name.

Automatically Generated IDs

PostgreSQL can generate ID values automatically through an identity column.

SQL

```
employee_id integer
GENERATED ALWAYS AS IDENTITY
PRIMARY KEY
```

 When a row is inserted, PostgreSQL generates the next ID automatically. Identity columns are backed by an implicit sequence that supplies generated values.

Insert without supplying an ID

SQL

```
INSERT INTO employees (
    first_name,
    last_name
)
VALUES (
    'Fatma',
    'Elraey'
);
```

Relationships Between Tables

Relational databases connect tables through relationships.

Main relationship types

- One-to-one

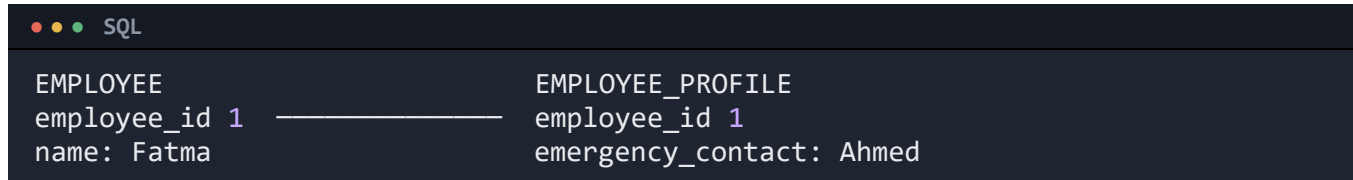
- One-to-many
- Many-to-many

Relationships reduce duplication and make the database easier to maintain.

One-to-One Relationship

Definition: one row in Table A is connected to one row in Table B.

Example: one employee has one employee profile.

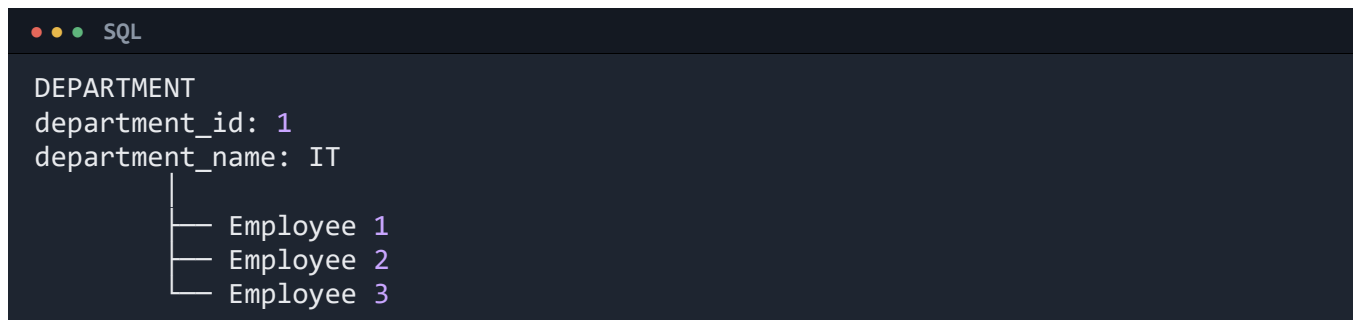


Possible use: separate frequently used employee information from sensitive profile information.

One-to-Many Relationship

Definition: one row in Table A can be connected to multiple rows in Table B.

Example: one department can contain many employees.



Database example

- One customer can place many orders.
- One teacher can teach many students.
- One category can contain many products.

Many-to-Many Relationship

Definition: multiple rows in Table A can be connected to multiple rows in Table B.

Example: an employee can work on many projects; a project can contain many employees.

Implementation: a third (junction) table is used.

employee_id	project_id
1	10
1	11
2	10

Section 2: DBMS and SQL Fundamentals

What Is a DBMS?

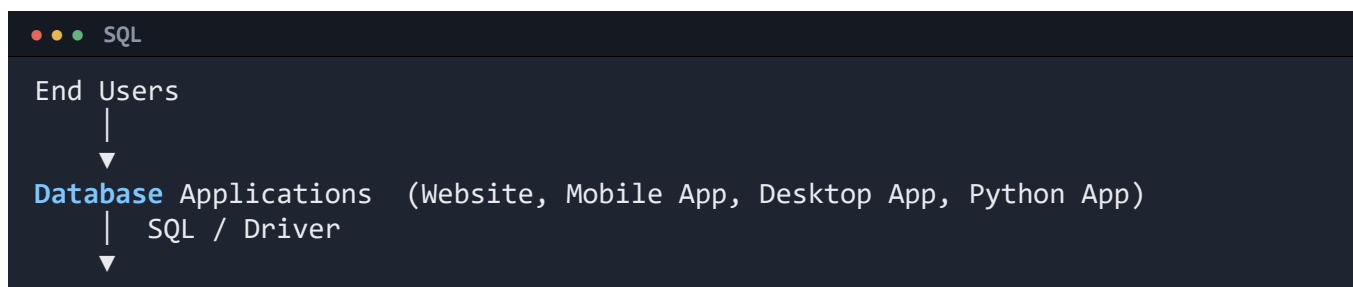
A Database Management System, or DBMS, is software used to create, organize, access, modify, secure, and maintain databases.

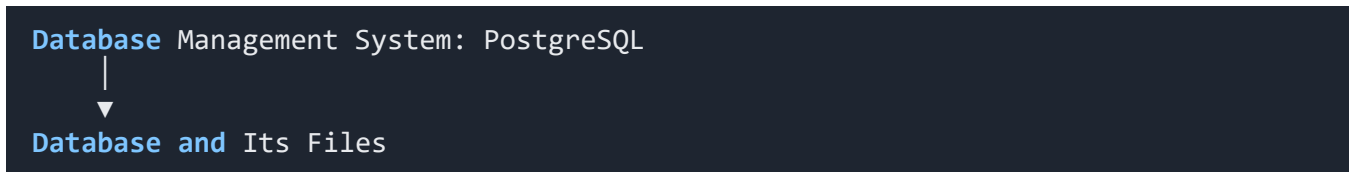
DBMS responsibilities

- Creating databases and tables
- Storing data
- Retrieving data
- Updating data
- Deleting data
- Managing users
- Enforcing constraints
- Controlling concurrent access
- Managing transactions
- Supporting backup and recovery

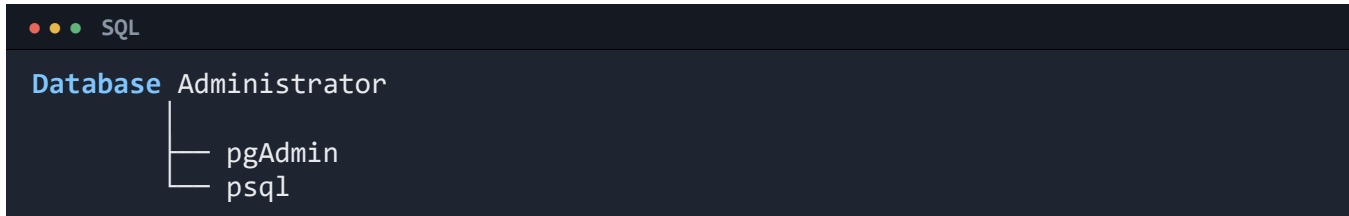
PostgreSQL is a DBMS.

Components of a Database System





Administrator access:



Multiple Users and One Central Database

A DBMS allows many users and applications to work with a central database.

Example — a hospital system

- Reception adds patient information.
- Doctors read medical records.
- Nurses update observations.
- The pharmacy reads prescriptions.
- Billing staff process payments.

All users interact with the same central database through controlled access.

Benefits

- A single source of truth
- Reduced duplication
- Controlled permissions
- Consistent information
- Concurrent access
- Central backup and maintenance

Examples of Database Systems

Database technologies do not all follow the same model.

Relational database systems

- PostgreSQL

- MySQL
- Microsoft SQL Server
- Oracle Database
- SQLite

Document database

- MongoDB

Key-value and in-memory database

- Redis

Distributed wide-column database

- Cassandra

! *Important clarification: PostgreSQL, MongoDB, Redis, and Cassandra are all database technologies, but they store and organize information differently.*

What Is SQL?

SQL means Structured Query Language. SQL is used to communicate with relational database systems.

SQL can be used to:

- Create databases.
- Create tables.
- Insert rows.
- Read stored data.
- Update stored data.
- Delete rows.
- Filter and sort results.
- Manage permissions.
- Control transactions.

 *PostgreSQL uses and extends the SQL language.*

Main SQL Command Groups

DDL — Data Definition Language

Defines database structures.

```
SQL  
CREATE  
ALTER  
DROP  
TRUNCATE
```

DML — Data Manipulation Language

Changes table data.

```
SQL  
INSERT  
UPDATE  
DELETE
```

DQL — Data Query Language

Retrieves data.

```
SQL  
SELECT
```

Additional groups

- TCL: Transaction Control Language
- DCL: Data Control Language

Day 1 focuses mainly on DDL, DML, and basic SELECT queries.

Section 3: PostgreSQL and Its Tools

What Is PostgreSQL?

PostgreSQL is:

- An open-source database system
- A relational and object-relational DBMS
- A client/server database platform
- A system that uses and extends SQL

- Suitable for small and large applications
- Available on Windows, Linux, and macOS

 PostgreSQL is described by its project as a powerful open-source object-relational database system.

PostgreSQL Tool Ecosystem

PostgreSQL Server

The service that stores the databases, processes SQL commands, controls access, and returns query results.

pgAdmin 4

A graphical administration and development tool for PostgreSQL.

SQL Shell (psql)

 A terminal-based interface for executing SQL and PostgreSQL commands.

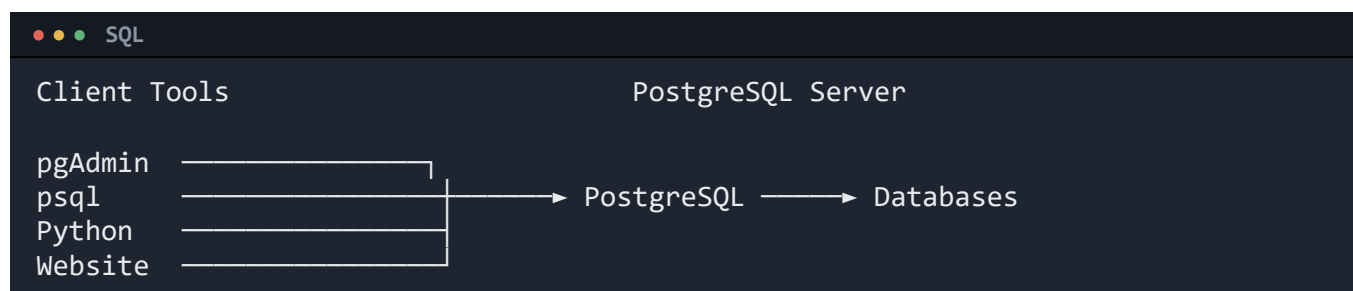
VS Code

A code editor that can be used to write SQL, Python, and application code.

Python database driver

A driver such as Psycopg allows a Python application to communicate with PostgreSQL.

Client/Server Architecture



Client: a program that sends a request. Server: PostgreSQL receives the request, processes it, and returns the result.



The client sends the SQL statement, and the server returns the matching rows.

PostgreSQL Version Used in the Course

Recommended course environment

- PostgreSQL 18.4
- pgAdmin 4
- SQL Shell
- Visual Studio Code
- Python in a later practical session

 *PostgreSQL 18.4 was released on May 14, 2026, and is the current maintenance release in the PostgreSQL 18 branch as of July 2026.*

 *Course-maintenance note: future course deliveries should use the latest supported stable minor release.*

Installing PostgreSQL on Windows

1. Open the official PostgreSQL download page.
2. Select Windows.
3. Select the EDB interactive installer.
4. Choose a supported PostgreSQL version.
5. Download and run the installer.
6. Select the required components.
7. Choose the installation directory.
8. Set a password for the postgres user.
9. Keep or change the server port.
10. Select the locale.
11. Complete the installation.

 *The official Windows page provides an EDB-certified installer and identifies StackBuilder as a tool for installing additional packages and drivers.*

Installer Components

PostgreSQL Server

Required to create and operate PostgreSQL databases.

pgAdmin 4

Graphical interface for administration and development.

Command Line Tools

Includes tools such as psql.

StackBuilder

An optional package manager for drivers, extensions, additional tools, and integration packages.

 *Recommendation: install the server, pgAdmin, and command-line tools. StackBuilder may be skipped initially and opened later.*

Installation Settings

Superuser

The default administrative role is commonly: postgres

Password

The password entered during installation is required when connecting.

Port

 *The customary PostgreSQL port is 5432. PostgreSQL normally uses port 5432 unless another port is configured.*

Locale

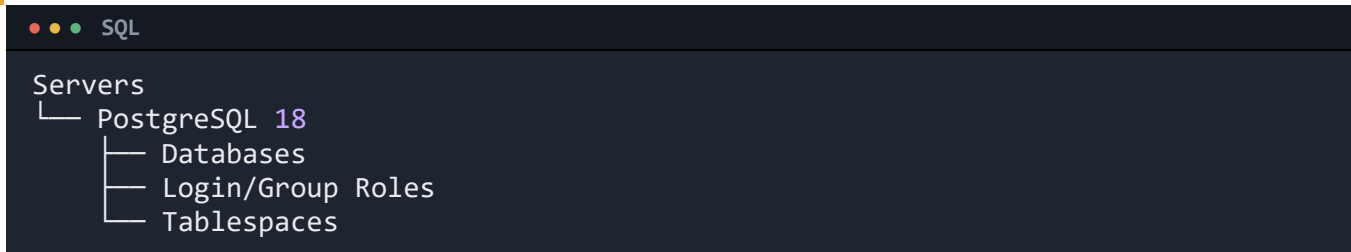
The locale controls language-related behavior such as sorting and character classification.

Opening pgAdmin 4

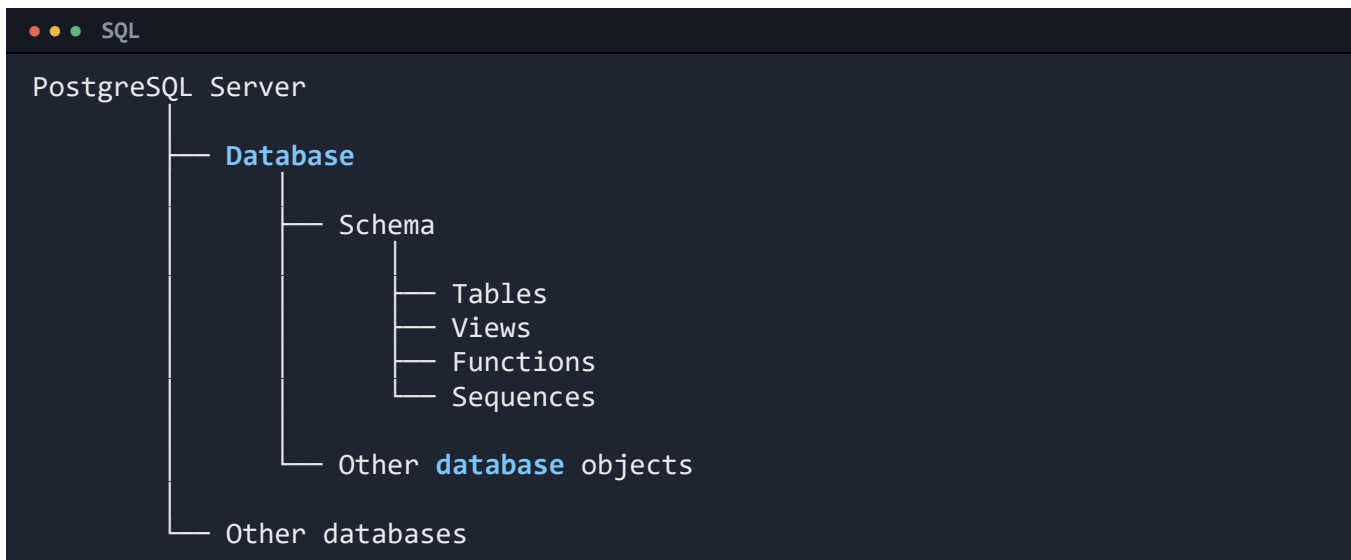
12. Start pgAdmin 4.
13. Enter the pgAdmin master password if requested.
14. Expand Servers in the Browser panel.
15. Select the registered PostgreSQL server.
16. Right-click the server.

17. Select Connect Server.
18. Enter the PostgreSQL password.
19. Expand the server tree.

Expected structure



PostgreSQL Object Hierarchy



Important terms

- One PostgreSQL server can manage multiple databases.
- A database can contain multiple schemas.
- A schema can contain multiple tables.
- A table contains rows and columns.

Default Databases

postgres

A default administrative connection database.

template1

A template used when creating new databases.

template0

A clean template used in special database-creation situations.

! *Important correction: do not run `CREATE DATABASE postgres`; the `postgres` database normally already exists. Create a new training database instead.*

Section 4: Creating and Deleting Databases

Creating a Database Using pgAdmin

20. Expand the PostgreSQL server.
21. Right-click Databases.
22. Select Create → Database.
23. Enter the database name (`day1_training`) and owner (`postgres`).
24. Keep the default settings.
25. Select Save.

Result: the new database appears under the Databases node.

Creating a Database Using SQL

SQL

```
CREATE DATABASE day1_training;
```

 *CREATE DATABASE is the command, `day1_training` is the new database name, and the semicolon ends the SQL statement. CREATE DATABASE creates a new PostgreSQL database and requires suitable privileges.*

Understanding pgAdmin's Generated Script

pgAdmin may generate a longer command:

SQL

```
CREATE DATABASE day1_training
WITH
OWNER = postgres
```

```
ENCODING = 'UTF8'
TABLESPACE = pg_default
CONNECTION LIMIT = -1
IS_TEMPLATE = false;
```

- OWNER: role that owns the database
- ENCODING: character encoding
- TABLESPACE: physical storage location
- CONNECTION LIMIT: maximum permitted connections
- -1: no specific database-level limit
- IS_TEMPLATE: whether the database can be used as a template

 *Locale note: values such as LC_COLLATE and LC_CTYPE depend on the operating system and installation locale. Do not copy locale values from another computer without checking compatibility.*

Adding a Database Comment

Comments document database objects.

```
SQL
COMMENT ON DATABASE day1_training
IS 'Training database for PostgreSQL Day 1';
```

Benefits

- Explains the purpose of an object
- Helps other developers
- Improves database documentation
- Can be viewed through management tools

SQL Shell: Essential Commands

Command	Purpose
\l	List databases
\c day1_training	Connect to a database
\conninfo	Display current connection
\dn	List schemas

Command	Purpose
\dt	List tables
\d employees	Describe a table
\q	Exit SQL Shell
\?	Show help for backslash commands
\h	Show SQL command help

 Backslash commands are processed by *psql*, while normal SQL statements are sent to the PostgreSQL server.

Creating a Database in SQL Shell

```
SQL
postgres=# CREATE DATABASE day1_training;
CREATE DATABASE
```

List the databases:

```
SQL
postgres=# \l
```

Connect to the new database:

```
SQL
postgres=# \c day1_training
You are now connected to database "day1_training".
```

Check the current connection:

```
SQL
day1_training=# \conninfo
```

Deleting a Database

```
SQL
DROP DATABASE day1_training;
```

Important: DROP DATABASE permanently removes the database, its tables, its rows, its functions, and its other database objects.

Safety recommendation:

```
SQL
```

```
DROP DATABASE IF EXISTS day1_training;
```

⚠ Database deletion is destructive and cannot normally be undone without a backup.

Why Can't We Drop the Current Database?

This command fails when connected to the target database:

```
SQL
```

```
day1_training=# DROP DATABASE day1_training;
ERROR: cannot drop the currently open database
```

Correct process — first connect to another database:

```
SQL
```

```
day1_training=# \c postgres
```

Then delete the target:

```
SQL
```

```
DROP DATABASE day1_training;
```

⚠ A database cannot be dropped through the session that is currently connected to it.

Section 5: Creating and Managing Tables

Database, Schema, and Table

After connecting to day1_training, the default schema is usually: public

The fully qualified table name contains: schema_name.table_name

Example: public.employees — public is the schema, employees is the table.

Recommended Naming Rules

Use

- Lowercase letters
- Meaningful names
- Underscores between words
- Plural names for tables

- Singular or descriptive names for columns

Recommended

SQL

```
employees
employee_id
first_name
last_name
hire_date
```

Avoid

SQL

```
"TB1"
"ID"
"FN"
"LS"
```

 PostgreSQL converts unquoted identifiers to lowercase. Quoted identifiers remain case-sensitive and require quotes every time they are used.

Creating the Employees Table

SQL

```
CREATE TABLE employees (
    employee_id integer
        GENERATED ALWAYS AS IDENTITY
        PRIMARY KEY,

    first_name varchar(50) NOT NULL,
    last_name  varchar(50) NOT NULL,
    phone     varchar(20),
    age       smallint,
    salary    numeric(10,2)
);
```

This statement defines the table name, the column names, the data type of each column, an automatically generated primary key, required name fields, and optional telephone, age, and salary fields.

Understanding the Table Definition

employee_id

SQL

```
employee_id integer
GENERATED ALWAYS AS IDENTITY
PRIMARY KEY
```

- Stores an integer.
- Is generated automatically.
- Uniquely identifies each employee.
- Cannot be NULL.

Names

```
SQL
```

```
first_name varchar(50) NOT NULL
last_name  varchar(50) NOT NULL
```

- Store text up to 50 characters.
- Must have a value.

Salary

```
SQL
```

```
salary numeric(10,2)
```

- Allows up to ten total digits.
- Keeps two digits after the decimal point.

Adding Validation with Constraints

A better table definition includes validation:

```
SQL
```

```
CREATE TABLE employees (
    employee_id integer
        GENERATED ALWAYS AS IDENTITY
        PRIMARY KEY,

    first_name varchar(50) NOT NULL,
    last_name  varchar(50) NOT NULL,
    phone      varchar(20),
    age        smallint CHECK (age >= 18),
    salary     numeric(10,2)
        CHECK (salary >= 0)
);
```

Constraints protect data quality

- PRIMARY KEY
- NOT NULL
- UNIQUE
- CHECK
- FOREIGN KEY
- DEFAULT

 PostgreSQL constraints apply rules that go beyond the basic restrictions provided by data types.

Important Correction: char Without a Length

The following definition is not appropriate for a person's name:

```
SQL
name char NOT NULL
```

Without a specified length, it produces a single-character column: character(1)

Better options

```
SQL
name varchar(100) NOT NULL
```

or:

```
SQL
name text NOT NULL
```

Creating a Table Through pgAdmin

26. Expand the database.
27. Expand Schemas.
28. Expand public.
29. Right-click Tables.
30. Select Create → Table.
31. Enter the table name.
32. Open the Columns section.
33. Add each column.
34. Select its data type.

35. Configure NOT NULL where required.
36. Configure the primary key.
37. Configure the ID column as an identity column.
38. Save the table.

 *Recommendation: for a new table, configure the identity property before inserting rows.*

Inspecting Tables

List the tables:

```
SQL
\dt
```

Describe a table:

```
SQL
\d employees
```

Display more detail:

```
SQL
\d+ employees
```

Query table information:

```
SQL
SELECT *
FROM employees;
```

At this stage, the result is empty because no rows have been inserted.

Deleting a Table

```
SQL
DROP TABLE employees;
```

Safer version:

```
SQL
DROP TABLE IF EXISTS employees;
```

 *DROP TABLE removes the table structure and the data inside it.*

pgAdmin method

39. Right-click the table.
40. Select Delete/Drop.
41. Review the generated SQL.
42. Confirm the action.

Section 6: Data Manipulation

CRUD Operations

CRUD represents four fundamental data operations.

CRUD Operation	SQL Command
Create a row	INSERT
Read rows	SELECT
Update rows	UPDATE
Delete rows	DELETE

Important distinction: CREATE TABLE creates a database object; INSERT creates a data row.

Inserting One Row

Because `employee_id` is generated automatically, do not include it in the statement.

```
SQL
INSERT INTO employees (
  first_name,
  last_name,
  phone,
  age,
  salary
)
VALUES (
  'Fatma',
  'Elraey',
  '01020304050',
  24,
  50000
);
```

Expected message:

```
SQL
```

```
INSERT 0 1
```

The final number indicates that one row was inserted.

Inserting and Returning the New Row

PostgreSQL can immediately return the inserted row:

```
SQL
```

```
INSERT INTO employees (
    first_name,
    last_name,
    phone,
    age,
    salary
)
VALUES (
    'Fatma',
    'Elraey',
    '01020304050',
    24,
    50000
)
RETURNING *;
```

 The `RETURNING` clause is useful for viewing values generated automatically, such as an identity value.

Inserting Multiple Rows

```
SQL
```

```
INSERT INTO employees (
    first_name,
    last_name,
    phone,
    age,
    salary
)
VALUES
    ('Ahmed', 'Hassan', '01030405060', 30, 60000),
    ('Sara', 'Ali', '01112131415', 28, 55000),
    ('Mariam', 'Mostafa', '01213141516', 35, 70000);
```

Expected message:

```
SQL
```

```
INSERT 0 3
```

💡 One `INSERT` statement can add multiple rows.

Why List Column Names?

Recommended

```
SQL

INSERT INTO employees (
    first_name,
    last_name,
    age
)
VALUES (
    'Ali',
    'Mahmoud',
    27
);
```

Less safe

```
SQL

INSERT INTO employees
VALUES (...);
```

Benefits of listing columns

- Makes the query easier to read.
- Prevents mistakes when table structure changes.
- Makes the relationship between values and columns clear.
- Allows optional columns to be omitted.

Retrieving All Rows

```
SQL

SELECT *
FROM employees;
```

- `SELECT` retrieves data.
- `*` means all columns.
- `FROM employees` identifies the source table.

Example output

employee_id	first_name	last_name	phone	age	salary
1	Fatma	Elraey	01020304050	24	50000.00
2	Ahmed	Hassan	01030405060	30	60000.00

Selecting Specific Columns

Instead of retrieving every column:

```
SQL
SELECT
    employee_id,
    first_name,
    last_name
FROM employees;
```

Benefits

- Clearer output
- Less unnecessary data
- Easier application processing
- Better query readability

Updating Data

```
SQL
UPDATE employees
SET salary = 65000
WHERE employee_id = 2;
```

Update multiple columns:

```
SQL
UPDATE employees
SET
    phone = '01199999999',
    salary = 68000
WHERE employee_id = 2;
```

Check the result:

```
SQL
SELECT *
FROM employees
WHERE employee_id = 2;
```

Always Check the WHERE Clause

Dangerous query — this changes the salary of every employee:

```
SQL
UPDATE employees
SET salary = 65000;
```

Safer query:

```
SQL
UPDATE employees
SET salary = 65000
WHERE employee_id = 2;
```

Recommended workflow

43. Write the condition as a SELECT.
44. Confirm the correct rows.
45. Change SELECT to UPDATE or DELETE.

Deleting Rows

Delete one employee:

```
SQL
DELETE FROM employees
WHERE employee_id = 5;
```

Delete using another condition:

```
SQL
DELETE FROM employees
WHERE first_name = 'Ahmed';
```

Check the remaining rows:

```
SQL
SELECT *
FROM employees;
```

Warning: without a WHERE clause, all rows are deleted:

```
SQL
DELETE FROM employees;
```

DELETE, TRUNCATE, and DROP

Command	Effect
DELETE	Deletes selected rows or all rows
TRUNCATE	Quickly removes all rows
DROP TABLE	Removes the table and its data

 SQL


```
DELETE FROM employees
WHERE employee_id = 4;
```

 SQL


```
TRUNCATE TABLE employees;
```

 SQL


```
DROP TABLE employees;
```

Editing Data Through pgAdmin

46. Right-click the table.
47. Select View/Edit Data.
48. Choose All Rows.
49. Add or modify values in the grid.
50. Select Save Data Changes.

 *Teaching point: the graphical interface is convenient, but SQL scripts are easier to repeat, review, automate, and keep in version control.*

Section 7: SQL Comments and Operators

SQL Comments

Comments explain SQL code and are not executed.

Single-line comment

 SQL


```
-- Retrieve all employees
SELECT *
```

```
FROM employees;
```

Multiline comment

```
SQL
```

```
/*
This query retrieves employees
whose salary is greater than 50,000.
*/
SELECT *
FROM employees
WHERE salary > 50000;
```

Uses of comments

- Documentation
- Explaining complex logic
- Temporarily disabling statements
- Separating sections of a script

Arithmetic Operators

Operator	Meaning	Example
+	Addition	2 + 5
-	Subtraction	5 - 2
*	Multiplication	2 * 5
/	Division	10 / 5
%	Remainder	14 % 4
^	Exponentiation	2 ^ 3
/	Square root	/ 25.0
/	Cube root	/ 27.0



PostgreSQL provides these numeric operators, including square-root and cube-root operators.

Arithmetic Examples

SQL

```
SELECT 2 + 5 AS result;
-- Result: 7
```

SQL

```
SELECT 14 % 4 AS remainder;
-- Result: 2
```

SQL

```
SELECT 2 ^ 3 AS power_result;
-- Result: 8
```

SQL

```
SELECT |/ 25.0 AS square_root;
-- Result: 5
```

Integer Division

Integer division removes the fractional part.

SQL

```
SELECT 5 / 2;
-- Result: 2
```

To receive a decimal result, use a decimal value:

SQL

```
SELECT 5.0 / 2;
-- Result: 2.5000000000000000
```



For integral types, PostgreSQL division truncates the result toward zero.

Section 8: Filtering Data

The WHERE Clause

The WHERE clause filters rows according to a condition.

SQL

```
SELECT column_list
FROM table_name
WHERE condition;
```

Example:

 SQL

```
SELECT *
FROM employees
WHERE employee_id = 3;
```

Only the employee whose ID is 3 is returned.

Comparison Operators

Operator	Meaning
=	Equal
<	Less than
>	Greater than
<=	Less than or equal
>=	Greater than or equal
<>	Not equal
!=	Not equal alias

 *<> is the standard SQL notation for “not equal,” while PostgreSQL also accepts !=.*

 SQL

```
SELECT *
FROM employees
WHERE age >= 30;
```

 SQL

```
SELECT *
FROM employees
WHERE salary <> 55000;
```

Creating Sample Employee Data

 SQL

```
INSERT INTO employees (
    first_name,
    last_name,
    phone,
    age,
    salary
)
```

VALUES

```
( 'Alice', 'Stone', '01000000001', 25, 5000),
( 'Bob', 'Smith', '01000000002', 30, 6000),
( 'Charlie', 'Brown', '01000000003', 28, 5500),
( 'David', 'Jones', '01000000004', 35, 7000),
( 'Emma', 'Wilson', '01000000005', 27, 5200),
( 'Frank', 'Taylor', '01000000006', 40, 8000),
( 'Grace', 'Clark', '01000000007', 32, 6500),
( 'Henry', 'Lewis', '01000000008', 29, 5800),
( 'Ivy', 'Walker', '01000000009', 26, 5100),
( 'Jack', 'Hall', '01000000010', 38, 7500);
```

This dataset will be used in the remaining examples.

Logical Operator: AND

AND requires all conditions to be true.

SQL

```
SELECT *
FROM employees
WHERE age > 27
      AND salary > 55000;
```

The query returns employees who are older than 27 and earn more than 55,000.

Logical Operator: OR

OR requires at least one condition to be true.

SQL

```
SELECT *
FROM employees
WHERE age > 27
      OR salary > 55000;
```

The query returns employees who are older than 27, or earn more than 55,000, or satisfy both conditions.

Logical Operator: NOT

NOT reverses a condition.

SQL

```
SELECT *
FROM employees
WHERE NOT age >= 30;
```

Equivalent form:

```
SQL
SELECT *
FROM employees
WHERE age < 30;
```

Another example:

```
SQL
SELECT *
FROM employees
WHERE first_name NOT LIKE 'A%';
```

Understanding NULL

NULL means unknown, missing, not provided, or not applicable. It is not zero, an empty string, or the word “NULL”.

Insert a missing value:

```
SQL
INSERT INTO employees (
    first_name,
    last_name,
    phone,
    age,
    salary
)
VALUES (
    'Ahmed',
    'Saleh',
    '01099999999',
    NULL,
    67000
);
```

Testing for NULL

Correct

```
SQL
SELECT *
FROM employees
WHERE age IS NULL;
```

Find non-NULL values:

```
SQL
```

```
SELECT *
FROM employees
WHERE age IS NOT NULL;
```

Incorrect

SQL

```
SELECT *
FROM employees
WHERE age = NULL;
```

✎ Use *IS NULL* or *IS NOT NULL*, not normal equality operators. PostgreSQL Boolean logic includes an unknown state represented by *NULL*.

Pattern Matching with LIKE

LIKE searches text using patterns.

Wildcard	Meaning
%	Zero or more characters
—	Exactly one character

Begins with A:

SQL

```
SELECT *
FROM employees
WHERE first_name LIKE 'A%';
```

Contains a:

SQL

```
SELECT *
FROM employees
WHERE first_name LIKE '%a%';
```

Ends with d:

SQL

```
SELECT *
FROM employees
WHERE first_name LIKE '%d';
```

LIKE Is Case-Sensitive

In PostgreSQL, these patterns can produce different results:

```
SQL
WHERE first_name LIKE 'A%'
WHERE first_name LIKE 'a%'
```

Case-insensitive search — use PostgreSQL's ILIKE operator:

```
SQL
SELECT *
FROM employees
WHERE first_name ILIKE 'a%';
```

 PostgreSQL supports SQL LIKE patterns and additional pattern-matching approaches.

Using the Underscore Wildcard

Second character is a:

```
SQL
SELECT *
FROM employees
WHERE first_name ILIKE '_a%';
```

Possible matches: David, Jack, Fatma.

Third character is a:

```
SQL
SELECT *
FROM employees
WHERE first_name ILIKE '__a%';
```

Possible matches: Charlie, Frank, Grace. Each underscore represents exactly one character.

The IN Operator

IN checks whether a value belongs to a list.

```
SQL
SELECT *
FROM employees
WHERE age IN (24, 25, 26, 32);
```

Equivalent but longer:

```
SQL
```

```
SELECT *
FROM employees
WHERE age = 24
      OR age = 25
      OR age = 26
      OR age = 32;
```

Excluding values:

```
SQL
SELECT *
FROM employees
WHERE age NOT IN (24, 25, 26, 32);
```

The BETWEEN Operator

BETWEEN checks whether a value is inside an inclusive range.

```
SQL
SELECT *
FROM employees
WHERE salary BETWEEN 50000 AND 60000;
```

This includes 50000, 60000, and every value between them.

Equivalent query:

```
SQL
SELECT *
FROM employees
WHERE salary >= 50000
      AND salary <= 60000;
```

Combining Conditions

```
SQL
SELECT
    employee_id,
    first_name,
    age,
    salary
FROM employees
WHERE age BETWEEN 25 AND 35
      AND salary >= 55000
      AND first_name ILIKE '%a%';
```

Query process

51. Select the required columns.
52. Read rows from employees.
53. Keep employees between 25 and 35.
54. Keep employees earning at least 55,000.
55. Keep employees whose names contain a.

Section 9: Working in pgAdmin

Opening the Query Tool

56. Select the required database.
57. Right-click the database.
58. Select Query Tool.
59. Write the SQL statement.
60. Select the Execute button or press F5.
61. Review Data Output, Messages, and Query History.



In the pgAdmin Query Tool, F5 executes the current query or selected script.

Executing Selected Statements

A Query Tool window may contain many statements:

```
SQL
SELECT *
FROM employees;

SELECT *
FROM employees
WHERE age >= 30;

SELECT *
FROM employees
WHERE salary > 60000;
```

To execute one statement, place the cursor inside the statement, or highlight the required statement, then execute it.



Good practice: execute and test SQL in small sections.

Reading PostgreSQL Messages

Message	Meaning
CREATE DATABASE	Successful database creation
CREATE TABLE	Successful table creation
INSERT 0 1	One inserted row
INSERT 0 3	Three inserted rows
DELETE 1	One deleted row
Query returned successfully	Successful query (pgAdmin)

Execution time varies by computer and is not a fixed expected value.

Common Beginner Errors

Missing semicolon

```
SQL
SELECT * FROM employees
```

The statement may remain incomplete in psql.

Using double quotes for string values

Incorrect:

```
SQL
WHERE first_name = "Ahmed"
```

Correct:

```
SQL
WHERE first_name = 'Ahmed'
```

Other common mistakes

- Storing a phone number as an integer removes leading zeros.
- Using = NULL instead of IS NULL.
- Forgetting the WHERE clause, which may update or delete every row.
- Connecting to the wrong database — use \conninfo to check.

Single and Double Quotes

Single quotes

Single quotes are used for text values:

```
SQL
'Ahmed'
'PostgreSQL'
'01020304050'
```

Double quotes

Double quotes are used for quoted database identifiers:

```
SQL
"EmployeeID"
"FirstName"
```

Recommendation: use lowercase, unquoted identifiers:

```
SQL
employee_id
first_name
```

This avoids unnecessary case-sensitivity problems.

Section 10: Day 1 Review

What We Learned

- What a database is
- How relational tables organize data
- Rows, columns, cells, and primary keys
- One-to-one, one-to-many, and many-to-many relationships
- The role of a DBMS
- The difference between PostgreSQL, pgAdmin, and psql
- How to install and connect to PostgreSQL
- How to create databases and tables
- How to insert, select, update, and delete data
- How to filter rows using SQL operators

Command Summary

```
SQL
CREATE DATABASE day1_training;

CREATE TABLE employees (...);

INSERT INTO employees (...) VALUES (...);

SELECT * FROM employees;

UPDATE employees
SET salary = 65000
WHERE employee_id = 2;

DELETE FROM employees
WHERE employee_id = 5;

DROP TABLE employees;

DROP DATABASE day1_training;
```

Filtering Summary

```
SQL
WHERE age = 30

WHERE age >= 30

WHERE age <> 30

WHERE age > 27 AND salary > 55000

WHERE age > 27 OR salary > 55000

WHERE age IS NULL

WHERE first_name ILIKE 'a%'

WHERE age IN (24, 25, 26)

WHERE salary BETWEEN 50000 AND 60000
```

Knowledge Check

62. What is the difference between a database and a DBMS?

63. What is the difference between a row and a column?
64. Why should a primary key be unique?
65. Why should telephone numbers be stored as text?
66. What does an identity column do?
67. What is the difference between DELETE and DROP TABLE?
68. Why is a WHERE clause important?
69. What is the difference between LIKE and ILIKE?
70. How do we test for a missing value?
71. Is BETWEEN inclusive or exclusive?

Day 1 Practical Laboratory

Laboratory Objectives

During the lab, participants will:

- Create a PostgreSQL database.
- Connect through pgAdmin and psql.
- Create a table with appropriate data types.
- Define a primary key and identity column.
- Insert single and multiple rows.
- Run SELECT, UPDATE, and DELETE.
- Apply comparison and logical operators.
- Use NULL, LIKE, ILIKE, IN, and BETWEEN.
- Save the SQL work as a script.

Lab Scenario

You have been asked to create a small employee database for a company. The database must store:

- Employee ID
- First name
- Last name
- Telephone number
- Age

- Salary

Business rules

- Every employee must have a unique ID.
- The ID must be generated automatically.
- First and last names are required.
- Telephone numbers must preserve leading zeros.
- Employees must be at least 18 years old.
- Salary cannot be negative.

Lab Part A: Create the Database

Task 1 — create the database:

```
SQL
CREATE DATABASE company_training;
```

Task 2 — connect to it:

```
SQL
\c company_training
```

Task 3 — confirm the connection:

```
SQL
\conninfo
```

Lab Part B: Create the Table

```
SQL
CREATE TABLE employees (
    employee_id integer
        GENERATED ALWAYS AS IDENTITY
        PRIMARY KEY,

    first_name varchar(50) NOT NULL,
    last_name  varchar(50) NOT NULL,
    phone      varchar(20),
    age        smallint CHECK (age >= 18),
    salary     numeric(10,2)
        CHECK (salary >= 0)
);
```

Inspect the table:



```
\d employees
```

Lab Part C: Insert Data

Insert the following rows:

First Name	Last Name	Phone	Age	Salary
Alice	Stone	01000000001	25	50000
Bob	Smith	01000000002	30	60000
Charlie	Brown	01000000003	28	55000
David	Jones	01000000004	35	70000
Emma	Wilson	01000000005	27	52000
Frank	Taylor	01000000006	40	80000
Grace	Clark	01000000007	32	65000
Henry	Lewis	01000000008	29	58000
Ivy	Walker	01000000009	26	51000
Jack	Hall	01000000010	38	75000

Use one multiple-row INSERT statement.

Lab Part D: Basic Queries

Write SQL queries to:

72. Display all employee columns.
73. Display only employee ID, first name, and salary.
74. Display the employee whose ID is 3.
75. Display employees who are at least 30 years old.
76. Display employees whose salary is greater than 60,000.
77. Display employees whose salary is not 55,000.
78. Display employees between 25 and 35 years old.

Lab Part E: Logical Operators

Write queries to display:

79. Employees older than 27 and earning more than 55,000.
80. Employees older than 35 or earning at least 75,000.
81. Employees whose age is 25, 28, or 32.
82. Employees whose salary is between 50,000 and 60,000.
83. Employees who are not younger than 30.

Lab Part F: Text Searching

Write queries to display employees whose first names:

84. Begin with A.
85. Begin with a, regardless of case.
86. End with d.
87. Contain the letter a.
88. Have a as their second character.
89. Do not begin with J.

Use both LIKE and ILIKE where appropriate.

Lab Part G: Missing Values

Insert two additional employees:

Employee 1

- First name: Fatma
- Last name: Ali
- Age: 24
- Salary: Missing

Employee 2

- First name: Ahmed
- Last name: Saleh
- Age: Missing
- Salary: 67,000

Then write queries to:

90. Display employees whose salary is NULL.

91. Display employees whose age is NULL.
92. Display employees whose salary is not NULL.

Lab Part H: Update and Delete

Update tasks

93. Change Bob's salary to 63,000.
94. Change Emma's telephone number.
95. Increase Frank's salary by 5,000.

Example:

```
SQL
UPDATE employees
SET salary = salary + 5000
WHERE first_name = 'Frank';
```

Delete tasks

96. Delete the employee whose ID is 5.
97. Delete employees whose salary is NULL.
98. Display the remaining rows.

Lab Safety Exercise

Before executing:

```
SQL
DELETE FROM employees
WHERE salary < 52000;
```

First execute:

```
SQL
SELECT *
FROM employees
WHERE salary < 52000;
```

? Question: why should the SELECT statement be tested before executing the DELETE statement?

 Expected answer: it confirms exactly which rows will be affected and reduces the risk of deleting the wrong data.

Practical Challenge

Create a second table named departments.

Required columns

- department_id
- department_name
- location

Requirements

- Automatically generated primary key
- Department name is required
- Department name must be unique
- Location is optional

Insert these departments

- Information Technology — Cairo
- Human Resources — Giza
- Finance — Alexandria
- Marketing — Cairo

Queries

99. Display all departments.
100. Display departments located in Cairo.
101. Change the Finance location to Cairo.
102. Delete the Marketing department.
103. Display the remaining departments.